Министерство образования Российской Федерации

Пензенский государственный университет

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №8

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Обход графа в ширину»

Выполнил: ст. гр. 25ВВВ1
Какулин В.С.
Самсонов М.А.
Проверил: к.т.н. доцент
Юрова О.В.
к.т.н., ст. преподователь
Деев М.В.

Пенза 2026

Название:

Обход графа в ширину.

Цель работы:

Изучить алгоритм обхода графа в ширину (BFS), реализовать его с использованием стандартного класса queue и собственной структуры данных «очередь» на основе связного списка, а также для матричной и списочной форм представления графа.

Лабораторное задание:

Задание 1:

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G. Выведите матрицу на экран.

2. Для сгенерированного графа осуществите процедуру обхода в ширину, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс queue из стандартной библиотеки C++.

3. Реализуйте процедуру обхода в ширину для графа, представленного списками смежности.

Задание 2:

1. Для матричной формы представления графов реализуйте алгоритм обхода в ширину с использованием очереди, построенной на основе структуры данных «список», самостоятельно созданной в лабораторной работе № 3.

2. Оцените время работы двух реализаций алгоритмов обхода в ширину (использующего стандартный класс queue и использующего очередь, реализованную самостоятельно) для графов разных порядков.

Ход работы:

Описание используемых алгоритмов:

Обход графа в ширину — один из распространённых способов обхода графа, основная идея которого состоит в том, чтобы посещать вершины по уровням удалённости от исходной вершины (где удалённость понимается как количество рёбер, по которым необходимо пройти до достижения вершины). Для соблюдения порядка посещения вершин по уровням используется структура данных «очередь»: при просмотре текущей вершины все её ещё не посещённые соседи помещаются в конец очереди, а на следующей итерации текущей становится вершина, стоящая в очереди первой. Чтобы не заходить повторно в уже пройденные вершины, они помечаются как посещённые.

∘ generate_adjacency_matrix(size, edge_percent): динамически выделяет матрицу size × size и заполняет её: для каждой пары i < j ребро добавляется с вероятностью edge_percent процентов (rand() % 100 < edge_percent), при этом G[i][j] = G[j][i], диагональ равна нулю.

∘ print_matrix(G, size): выводит матрицу смежности с заголовками строк и столбцов.

∘ createNode / push_node / pop_node: универсальные функции для работы со связным списком (struct Node). createNode выделяет память под новый узел, push_node добавляет вершину в конец списка (используется как при формировании списков смежности, так и для добавления элемента в пользовательскую очередь), pop_node удаляет первый элемент из головы списка и освобождает память.

∘ print_lists(adjLists, size): выводит списки смежности для каждой вершины графа.

∘ bfs_matrix_queue(v, size, G, NUM): реализация обхода в ширину по матрице смежности с использованием стандартного класса queue из библиотеки C++. Помещает стартовую вершину v в очередь и помечает её как посещённую (NUM[v] = true). В цикле, пока очередь не пуста, извлекает первую вершину, выводит её номер, затем просматривает строку G[v] и каждую смежную непосещённую вершину добавляет в очередь и помечает как посещённую.

∘ run_task_1(size, G, start): обход в ширину по матрице смежности (std::queue). Создаёт массив посещённых вершин, запускает bfs_matrix_queue от стартовой вершины, а затем — от всех оставшихся непосещённых вершин.

∘ bfs_list_queue(v, adjLists, NUM): обход в ширину по спискам смежности с использованием std::queue. Аналогичен bfs_matrix_queue, но для поиска смежных вершин перебирает не строку матрицы, а связный список соседей текущей вершины.

∘ run_task_1_lists(size, G, start): строит списки смежности из матрицы смежности, выводит их на экран, выполняет обход в ширину (bfs_list_queue), выводит порядок вершин и освобождает выделенную под списки память.

∘ bfs_custom_queue(v, size, G, NUM): реализация обхода в ширину по матрице смежности с использованием собственной очереди, построенной на основе односвязного списка (struct Node). Вместо методов стандартного контейнера использует push_node для помещения вершины в конец очереди и pop_node для извлечения первой вершины из начала очереди.

∘ run_task_2(size, G, start): обход в ширину по матрице смежности с использованием пользовательской очереди. Создаёт массив посещённых вершин, запускает bfs_custom_queue от стартовой и всех оставшихся непосещённых вершин, после чего освобождает память.

Результат выполнения работы:

```
Введите количество вершин графа: 20
Введите вероятность появления ребра в процентах (от 0 до 100): 56
Выводить матрицу, списки смежности и порядок обхода? (1 – да, 0 – нет): 1

Сгенерированная матрица смежности G (20 вершин, вероятность ребра 56%):
    v0  v1  v2  v3  v4  v5  v6  v7  v8  v9  v10 v11 v12 v13 v14 v15 v16 v17 v18 v19
v0   0   0   1   0   0   1   0   0   1   0   0   0   0   0   1   0   1   0   0   1
v1   0   0   0   1   1   1   0   1   1   1   0   1   0   0   1   1   1   1   1   1
v2   1   0   0   0   1   1   1   0   0   1   0   0   0   1   1   1   1   1   1   0
v3   0   1   0   0   1   1   0   0   0   0   0   0   0   0   1   1   1   0   1   0
v4   0   1   1   1   0   1   0   0   1   0   1   1   0   0   1   1   1   0   0   0
v5   1   1   1   1   1   0   1   1   1   0   0   1   1   0   0   0   0   1   0   0
v6   0   0   1   0   0   1   0   1   1   0   0   0   1   0   1   1   1   0   0   1
v7   0   1   0   0   0   1   1   0   0   1   0   1   0   1   1   0   1   1   0   0
v8   1   1   0   0   1   1   1   0   0   1   1   1   1   0   1   1   1   0   0   1
v9   0   1   1   0   0   0   0   1   1   0   1   1   0   1   1   1   0   1   0   0
v10  0   0   0   0   1   0   0   0   1   1   0   1   1   1   0   1   1   0   1   0
v11  0   1   0   0   1   1   0   1   1   1   1   0   1   0   1   0   0   1   0   1
v12  0   0   0   0   0   1   1   0   1   0   1   1   0   1   1   1   1   1   0   0
v13  0   0   1   0   0   0   0   1   0   1   1   0   1   0   1   1   0   0   1   1
v14  1   1   1   1   1   0   1   1   1   1   0   1   1   1   0   0   0   0   0   1
v15  0   1   1   1   1   0   1   0   1   1   1   0   1   1   0   0   1   1   0   1
v16  1   1   1   1   1   0   1   1   1   0   1   0   1   0   0   1   0   0   1   0
v17  0   1   1   0   0   1   0   1   0   1   0   1   1   0   0   1   0   0   0   1
v18  0   1   1   1   0   0   0   0   0   0   1   0   0   1   0   0   1   0   0   0
v19  1   1   0   0   0   0   1   0   1   0   0   1   0   1   1   1   0   1   0   0
Введите начальную вершину (от 0 до 19): 13

Списки смежности графа:
v0: 2 5 8 14 16 19
v1: 3 4 5 7 8 9 11 14 15 16 17 18 19
v2: 0 4 5 6 9 13 14 15 16 17 18
v3: 1 4 5 14 15 16 18
v4: 1 2 3 5 8 10 11 14 15 16
v5: 0 1 2 3 4 6 7 8 11 12 17
v6: 2 5 7 8 12 14 15 16 19
v7: 1 5 6 9 11 13 14 16 17
v8: 0 1 4 5 6 9 10 11 12 14 15 16 19
v9: 1 2 7 8 10 11 13 14 15 17
v10: 4 8 9 11 12 13 15 16 18
v11: 1 4 5 7 8 9 10 12 14 17 19
v12: 5 6 8 10 11 13 14 15 16 17
v13: 2 7 9 10 12 14 15 18 19
v14: 0 1 2 3 4 6 7 8 9 11 12 13 19
v15: 1 2 3 4 6 8 9 10 12 13 16 17 19
v16: 0 1 2 3 4 6 7 8 10 12 15 18
v17: 1 2 5 7 9 11 12 15 19
v18: 1 2 3 10 13 16
v19: 0 1 6 8 11 13 14 15 17

ЗАДАНИЕ №1: Использование очереди из стандартной библиотеки queue

Обход в ширину (Матрица смежности)
Стартовая вершина: 13
Порядок обхода вершин: 13 2 7 9 10 12 14 15 18 19 0 4 5 6 16 17 1 11 8 3
Время выполнения: 0.000028 сек.

Обход в ширину (Списки смежности)
Стартовая вершина: 13
Порядок обхода вершин: 13 2 7 9 10 12 14 15 18 19 0 4 5 6 16 17 1 11 8 3
Время выполнения: 0.000058 сек.

ЗАДАНИЕ №2: Использование пользовательской очереди

Обход в ширину (Матрица смежности)
Стартовая вершина: 13
Порядок обхода вершин: 13 2 7 9 10 12 14 15 18 19 0 4 5 6 16 17 1 11 8 3
Время выполнения: 0.000020 сек.

Обход в ширину (Списки смежности)
Стартовая вершина: 13
Порядок обхода вершин: 13 2 7 9 10 12 14 15 18 19 0 4 5 6 16 17 1 11 8 3
Время выполнения: 0.000016 сек.
```

Рисунок 1 – результат выполнения программы при количестве вершин равным 20

Оценка времени работы алгоритмов обхода в ширину:

Таблица 1. Время обхода в ширину (N – количество вершин, p – вероятность появления ребра)

N	p, %	Матрица, std::queue (сек.)	Списки, std::queue (сек.)	Матрица, польз. очередь (сек.)	Списки, польз. очередь (сек.)
500	1	0.000179	0.000029	0.000209	0.000119
500	10	0.000487	0.000082	0.000548	0.000547
500	50	0.001284	0.000200	0.001135	0.004029
1000	1	0.000841	0.000201	0.001069	0.000887
1000	10	0.001863	0.000307	0.002225	0.005248
1000	50	0.005274	0.000735	0.004531	0.022296
2000	1	0.003121	0.000217	0.005014	0.003340
2000	10	0.007808	0.000952	0.009351	0.018615
2000	50	0.021875	0.003345	0.018414	0.170482
3000	1	0.010078	0.000784	0.014296	0.011336
3000	10	0.017679	0.001991	0.021514	0.056934
3000	50	0.046515	0.007092	0.041495	0.366554

Время обхода по матрице смежности растёт как $O(N^2)$ и почти не зависит от количества рёбер, потому что для каждой вершины просматривается вся строка матрицы длиной N. Это видно по замерам: при увеличении N в 3 раза (с 1000 до 3000) время растёт примерно в 9 раз.

Время обхода по спискам смежности растёт как O(N + E), где E — число

рёбер, потому что для каждой вершины просматриваются только её соседи. Поэтому выигрыш списков сильнее всего проявляется на разреженных графах. Со стандартной очередью при p = 1% и N = 3000 списки работают примерно в 13 раз быстрее матрицы, при p = 10% — в 9 раз, при p = 50% — в 6,5 раза. С ростом плотности число рёбер приближается к N^2, и преимущество списков уменьшается.

Списки смежности с пользовательской очередью быстрее матрицы только на разреженном графе (p = 1%), а при p = 10% и p = 50% медленнее в 2,5–9 раз. У этого две причины:

узлы связного списка разбросаны по памяти, и каждый переход по указателю next может приводить к промаху кэша, тогда как строка матрицы лежит в памяти последовательно и читается быстро;

функция добавления элемента в пользовательскую очередь каждый раз проходит весь список до последнего элемента, поэтому её сложность O(n), а не O(1). Чем плотнее граф, тем длиннее очередь и тем дороже каждое добавление.

При работе с матрицей смежности стандартная и пользовательская очереди показывают близкое время: основная часть времени уходит на просмотр строк матрицы, а не на операции с очередью.

Таким образом, списки смежности эффективнее матрицы для разреженных графов, где E значительно меньше N^2. Для плотных графов асимптотический выигрыш списков невелик, и на первый план выходят накладные расходы конкретной реализации: последовательное расположение матрицы в памяти позволяет ей конкурировать со списками и даже обгонять их.

Вывод:

В ходе выполнения лабораторной работы был изучен алгоритм обхода графа в ширину. Реализованы три варианта обхода: по матрице смежности с использованием стандартного класса queue, по спискам смежности и по матрице смежности с использованием пользовательской очереди на основе связного списка. Все реализации дают одинаковый порядок обхода вершин, корректно обрабатывают несвязные графы.